

UNIVERSITY OF ASIA PACIFIC



Course Code: CSE 430
Course Title: Compiler Lab

An Interactive GUI-Based Nabila's Compiler *A Complete Implementation of Six-Phase Compiler Pipeline with LL(1) Grammar Transformation*

Prepared for:
Md. Shafiqul Islam, Ph.D.
Ass.Professor , Dept. of CSE
University of Asia Pacific

Prepared by:
Anika Nower Nabila
ID No: 22101152
Sec: D

Department of Computer Science and Engineering
University of Asia Pacific

Table of Contents

Abstract	3
1. System Overview	3
1.1 Technology Stack.....	3
2. Compilation Pipeline — Block Diagram.....	4
3. LL(1) Grammar Transformation Flow.....	4
4. Module Interaction Diagram.....	4
5. Project File Structure.....	5
6. Source Code Listings	6
6.1 Lexical Analyzer (lexer.py)	6
6.2 Syntax Processor (parser.py)	10
6.3 Symbol Table (symbol_table.py).....	17
6.4 Semantic Analyser (semantic.py)	22
6.5 Three-Address Code Generator (tac.py).....	24
6.6 Assembly Code Generator (asm.py).....	29
6.7 Grammar Utilities (grammar_utils.py)	35
6.7.1 Grammar Normaliser	35
6.7.2 Left Recursion Removal	36
6.7.3 Left Factoring.....	37
6.7.4 FIRST and FOLLOW Sets	39
8. Test Cases & Expected Output	43
7. Phase Summary.....	49
8. References	50

Abstract

Nabila’s compiler is a fully functional, GUI-driven Mini C Compiler built entirely in Python, designed to demonstrate the complete workflow of modern compiler construction. The system processes C-like source code through six sequential compilation phases: Lexical Analysis, Syntax Analysis, Symbol Table Construction, Semantic Analysis, Intermediate Representation (Three-Address Code) Generation, and x86 Assembly Code Generation.

Unlike typical academic compiler projects, Nabila’s compiler is built from scratch without relying on external parser generators such as PLY. The lexer (`lexer.py`) is a hand-written character-level scanner; the parser (`parser.py`) is a hand-written recursive-descent parser that produces an Abstract Syntax Tree (AST). The GUI is built with PyQt5 and presents each phase in a dedicated tab, allowing users to observe the full compilation process in real time.

Nabila’s compiler also includes a standalone LL(1) Grammar Transformation module (`grammar_utils.py`, `grammar_tool.py`) capable of left recursion elimination, left factoring, and FIRST/FOLLOW set computation. The project serves as both an educational platform and a practical demonstration of compiler engineering.

1. System Overview

Nabila’s compiler accepts C-like source code through a PyQt5-based IDE-style interface and compiles it through six independent, modular phases. Each phase is implemented in its own Python file and passes structured data to the next stage. The GUI (`gui.py`) acts as the central coordinator: it invokes each phase in sequence and renders the output in dedicated tabs.

A key architectural decision is that every module is self-contained. The lexer produces a list of token dictionaries, the parser consumes that list and produces an AST, the symbol table builder walks the AST, the semantic analyser inspects the AST with the symbol table, the TAC generator emits intermediate code from the AST, and the assembly generator translates TAC into pseudo-x86 Intel-syntax instructions.

1.1 Technology Stack

Component	Technology / Approach
Language	Python 3.10+
Lexer	Hand-written character-level scanner (<code>lexer.py</code>)
Parser	Hand-written recursive-descent parser producing an AST (<code>parser.py</code>)
GUI	PyQt5 with optional PyQtWebEngine; launched via <code>main.py</code>
Intermediate Representation	Three-Address Code (TAC) — plain string list (<code>tac.py</code>)
Assembly Generator	Pseudo x86 Intel syntax, 32-bit (<code>asm.py</code>)
LL(1) Analysis	Custom <code>grammar_utils.py</code> (no external dependencies)

2. Compilation Pipeline — Block Diagram

When the user clicks Run in the Nabila’s compiler IDE, the source code travels through a six-phase pipeline. Each phase performs a distinct task and hands its output to the next phase. The pipeline below shows the module responsible, its input, and its output at every stage.

Phase	Module	Input	Output
1 — Lexical Analysis	lexer.py	Raw source string	List of token dicts {type, val}
2 — Syntax Analysis	parser.py	Token list	AST (nested dict tree)
3 — Symbol Table	symbol_table.py	AST	Scoped symbol dictionary + errors
4 — Semantic Analysis	semantic.py	AST + symbol data	List of semantic error strings
5 — IR / TAC Generation	tac.py	AST	List of TAC instruction strings
6 — Assembly Generation	asm.py	TAC string list	x86 Intel-syntax assembly lines

3. LL(1) Grammar Transformation Flow

The LL(1) Grammar Transformation module (grammar_utils.py + grammar_tool.py) is a standalone educational library integrated into Nabila’s compiler . It accepts context-free grammars entered by the user and automatically applies classical LL(1) preprocessing techniques.

Step	Function	Description
1	tokenize_production()	Splits a production string into atomic symbols (handles compact and spaced notation)
2	_normalize_grammar()	Ensures every token is a single atomic symbol; greedy-matches known NTs
3	remove_left_recursion()	Eliminates direct and indirect left recursion via ordered NT substitution; creates A' primes
4	left_factoring()	Groups productions by first symbol, extracts longest common prefix; iterates until stable
5	compute_first()	Fixed-point iteration to compute FIRST sets for all non-terminals
6	compute_follow()	Fixed-point iteration to compute FOLLOW sets; seeds start symbol with \$
7	compute_parsing_table()	Builds the LL(1) predictive parsing table and detects conflicts

4. Module Interaction Diagram

The diagram below shows how each Python module in Nabila’s compiler interacts with others during compilation. gui.py acts as the central controller. The lexer feeds the parser; the parser AST drives the symbol table, semantic analyser, TAC generator, and assembly generator. Grammar modules are independent.

Module	Depends On	Role
main.py	(entry point)	Checks/installs PyQt5, launches the GUI
gui.py	All phase modules	Central controller; renders output in tabs
lexer.py	(none)	Tokenises source code into {type, val} dicts
parser.py	lexer.py	Recursive-descent parser; builds AST
symbol_table.py	parser.py (AST)	Builds scoped symbol table; detects re-declarations
semantic.py	symbol_table.py, parser.py	Semantic checks; function arity, undeclared ids
tac.py	parser.py (AST)	Generates Three-Address Code string list
asm.py	tac.py	Translates TAC to pseudo-x86 Intel assembly
grammar_utils.py	(none)	LL(1) transformations and FIRST/FOLLOW computation
grammar_tool.py	grammar_utils.py	CLI interface for grammar transformations

5. Project File Structure

The project is divided into focused, single-responsibility modules. Each file implements exactly one compiler phase or support function.

File	Phase	Responsibility
main.py	Entry Point	Auto-installs dependencies, launches PyQt5 GUI
gui.py	GUI / Controller	PyQt5 IDE; coordinates all phases; renders per-tab output
lexer.py	Phase 1 — Lexer	Hand-written scanner; keywords, operators, literals, comments
parser.py	Phase 2 — Parser	Recursive-descent parser; produces AST; handles all C constructs
symbol_table.py	Phase 3 — Symbol Table	Scope stack; declares variables/functions; detects re-declarations
semantic.py	Phase 4 — Semantic Analysis	Reuses parser errors; adds arity checks; 5 semantic rules
tac.py	Phase 5 — TAC/IR	Walks AST; emits Three-Address Code strings with temps & labels
asm.py	Phase 6 — Assembly	Translates TAC to Intel-syntax pseudo-x86; back-patches frame size
grammar_utils.py	LL(1) Tool	Left recursion removal, left factoring, FIRST/FOLLOW, parsing table
grammar_tool.py	LL(1) CLI	CLI front-end for grammar_utils; handles compact and spaced input

6. Source Code Listings

6.1 Lexical Analyzer (lexer.py)

The lexer is a hand-written character-level scanner. It iterates over the source string with a single index pointer and classifies each token by examining individual characters. It does not use regular expression engines or PLY. Keywords are detected by checking identifiers against a fixed set. Two-character operators are tried before single-character ones.

Key Design Points

- Two-character operators (`==`, `!=`, `<=`, `>=`) are matched before single-char operators to avoid ambiguity.
- Block comments (`/* ... */`) and line comments (`//`) are stripped and skipped.
- Numbers are scanned digit-by-digit; a dot in the sequence produces `FLOAT_LITERAL` instead of `INT_LITERAL`.
- Unknown characters are silently skipped, allowing partial tokenisation of malformed input.
- Each token is a plain dict: `{ 'type': ..., 'val': ... }`.

Supported Token Types

Category	Token Types
Keywords	INT, FLOAT, VOID, RETURN, IF, ELSE, WHILE, CHAR
Identifiers	IDENTIFIER
Literals	INT_LITERAL, FLOAT_LITERAL, STRING_LITERAL
Operators	PLUS, MINUS, MUL, DIV, ASSIGN, EQ, NEQ, LT, GT, LE, GE
Punctuation	LPAREN, RPAREN, LBRACE, RBRACE, SEMICOLON, COMMA

```
# =====
# PHASE 1 - LEXER (Tokenizer)
# Mini C Compiler
# =====

KEYWORDS = {'int', 'float', 'void', 'return', 'if', 'else', 'while',
            'char'}

OPERATORS = {
    '==': 'EQ', '!=': 'NEQ', '<=': 'LE', '>=': 'GE',
    '<': 'LT', '>': 'GT', '=': 'ASSIGN',
    '+': 'PLUS', '-': 'MINUS', '*': 'MUL', '/': 'DIV'
}

PUNCTS = {
    '(': 'LPAREN', ')': 'RPAREN',
    '{': 'LBRACE', '}': 'RBRACE',
```

```

';': 'SEMICOLON', ',': 'COMMA'
}

def lex(src: str) -> list[dict]:
    tokens = []
    i = 0
    n = len(src)

    while i < n:
        # skip whitespace
        if src[i].isspace():
            i += 1
            continue

        # line comment
        if src[i:i+2] == '//':
            while i < n and src[i] != '\n':
                i += 1
            continue

        # block comment
        if src[i:i+2] == '/*':
            i += 2
            while i < n and src[i:i+2] != '*/':
                i += 1
            i += 2
            continue

        # string literal
        if src[i] == '"':
            s = ''
            i += 1
            while i < n and src[i] != '"':
                s += src[i]
                i += 1
            s += '"'
            i += 1
            tokens.append({'type': 'STRING_LITERAL', 'val': s})
            continue

```

```

# number
if src[i].isdigit():
    num = ''
    while i < n and (src[i].isdigit() or src[i] == '.'):
        num += src[i]
        i += 1
    tok_type = 'FLOAT_LITERAL' if '.' in num else
'INT_LITERAL'
    tokens.append({'type': tok_type, 'val': num})
    continue

# identifier / keyword
if src[i].isalpha() or src[i] == '_':
    ident = ''
    while i < n and (src[i].isalnum() or src[i] == '_'):
        ident += src[i]
        i += 1
    if ident in KEYWORDS:
        tokens.append({'type': ident.upper(), 'val': ident})
    else:
        tokens.append({'type': 'IDENTIFIER', 'val': ident})
    continue

# two-char operators
two = src[i:i+2]
if two in OPERATORS:
    tokens.append({'type': OPERATORS[two], 'val': two})
    i += 2
    continue

# single-char operators
if src[i] in OPERATORS:
    tokens.append({'type': OPERATORS[src[i]], 'val':
src[i]})
    i += 1
    continue

# punctuation
if src[i] in PUNCTS:
    tokens.append({'type': PUNCTS[src[i]], 'val': src[i]})
    i += 1

```



```

        continue

        i += 1 # unknown char, skip

    return tokens

def print_tokens(tokens: list[dict]):
    KW_TYPES =
    {'INT', 'FLOAT', 'VOID', 'RETURN', 'IF', 'ELSE', 'WHILE', 'CHAR'}
    LIT_TYPES = {'INT_LITERAL', 'FLOAT_LITERAL', 'STRING_LITERAL'}
    OP_TYPES =
    {'PLUS', 'MINUS', 'MUL', 'DIV', 'ASSIGN', 'EQ', 'NEQ', 'LT', 'GT', 'LE', 'GE'}
    PNC_TYPES =
    {'LPAREN', 'RPAREN', 'LBRACE', 'RBRACE', 'SEMICOLON', 'COMMA'}

    kw_count = sum(1 for t in tokens if t['type'] in KW_TYPES)
    id_count = sum(1 for t in tokens if t['type'] == 'IDENTIFIER')

    print("=" * 55)
    print(" PHASE 1 – LEXER / TOKEN STREAM")
    print("=" * 55)
    print(f" Total Tokens : {len(tokens)}")
    print(f" Keywords      : {kw_count}")
    print(f" Identifiers    : {id_count}")
    print("-" * 55)
    print(f" {'#':<5} {'Type':<20} {'Value'}")
    print("-" * 55)

    for idx, tok in enumerate(tokens, 1):
        t, v = tok['type'], tok['val']
        if t in KW_TYPES:
            label = '[KW]'
        elif t == 'IDENTIFIER':
            label = '[ID]'
        elif t in LIT_TYPES:
            label = '[LIT]'
        elif t in OP_TYPES:
            label = '[OP]'
        elif t in PNC_TYPES:
            label = '[PNC]'

```

```

    else:
        label = ''
    print(f" {idx:<5} {t:<20} {v!r:<15} {label}")

print("=" * 55)

```

6.2 Syntax Processor (parser.py)

The parser is a hand-written recursive-descent parser. It consumes the token list produced by the lexer and builds a nested dictionary AST. Each grammar rule maps directly to a Python function. Operator precedence is encoded structurally through the call hierarchy: comparisons call add/sub, which call mul/div, which call unary, which calls primary.

Key Design Points

- Mutable position pointer pos=[0] is shared across all parse functions via closure.
- peek() returns the current token without advancing; consume() advances and returns it.
- Operator precedence is managed entirely by the call hierarchy (no precedence table needed).
- ParseError is raised with descriptive messages on unexpected tokens.
- Functions handled: variable declarations, assignments, if/else, while, return, function defs, function calls.

```

# =====
# PHASE 2 – PARSER (builds Abstract Syntax Tree)
# Mini C Compiler
# =====

class ParseError(Exception):
    pass

def parse(tokens: list[dict]):
    pos = [0] # mutable pointer

    def peek():
        return tokens[pos[0]] if pos[0] < len(tokens) else {'type':
'EOF', 'val': ''}

    def consume(expected_type=None):
        t = tokens[pos[0]] if pos[0] < len(tokens) else {'type':
'EOF', 'val': ''}
        if expected_type and t['type'] != expected_type:
            raise ParseError(

```

```

        f"Expected {expected_type}, got {t['type']}"
    ('{t['val']}')
    )
    pos[0] += 1
    return t

def match(tok_type):
    if peek()['type'] == tok_type:
        pos[0] += 1
        return True
    return False

# — grammar rules —————

def parse_program():
    node = {'type': 'PROGRAM', 'children': []}
    while peek()['type'] != 'EOF':
        node['children'].append(parse_func_def())
    return node

def parse_type():
    t = peek()['type']
    if t in ('INT', 'FLOAT', 'VOID', 'CHAR'):
        pos[0] += 1
        return t.lower()
    raise ParseError(f"Expected type keyword, got {t}")

def parse_func_def():
    rtype = parse_type()
    name = consume('IDENTIFIER')['val']
    consume('LPAREN')
    params = {'type': 'PARAM_LIST', 'children': []}
    while peek()['type'] != 'RPAREN':
        pt = parse_type()
        pn = consume('IDENTIFIER')['val']
        params['children'].append({'type': 'PARAM', 'sval': pn,
                                   'data_type': pt, 'children':
[]})
        if not match('COMMA'):
            break
    consume('RPAREN')

```

```

        block = parse_block()
        return {'type': 'FUNC_DEF', 'sval': name, 'data_type':
rtype,
                'children': [params, block]}

def parse_block():
    consume('LBRACE')
    stmts = {'type': 'STMT_LIST', 'children': []}
    while peek()['type'] not in ('RBRACE', 'EOF'):
        s = parse_stmt()
        if s:
            stmts['children'].append(s)
    consume('RBRACE')
    return {'type': 'BLOCK', 'children': [stmts]}

def parse_stmt():
    t = peek()['type']
    if t in ('INT', 'FLOAT', 'VOID', 'CHAR'):
        return parse_var_decl_line()
    if t == 'RETURN':
        return parse_return()
    if t == 'IF':
        return parse_if()
    if t == 'WHILE':
        return parse_while()
    if t == 'LBRACE':
        return parse_block()
    if t == 'SEMICOLON':
        pos[0] += 1
        return None
    stmt = parse_expr_stmt()
    consume('SEMICOLON')
    return stmt

def parse_var_decl_line():
    dt = parse_type()
    stmts = []
    while True:
        name = consume('IDENTIFIER')['val']
        init = None
        if match('ASSIGN'):

```

```

        init = parse_expr()
        stmts.append({'type': 'VAR_DECL', 'sval': name,
'data_type': dt,
                    'children': [init] if init else []})
        if not match('COMMA'):
            break
        consume('SEMICOLON')
    if len(stmts) == 1:
        return stmts[0]
    return {'type': 'STMT_LIST', 'children': stmts}

def parse_return():
    consume('RETURN')
    children = []
    if peek()['type'] != 'SEMICOLON':
        children.append(parse_expr())
    consume('SEMICOLON')
    return {'type': 'RETURN', 'children': children}

def parse_if():
    consume('IF')
    consume('LPAREN')
    cond = parse_expr()
    consume('RPAREN')
    then = parse_stmt()
    children = [cond, then]
    if peek()['type'] == 'ELSE':
        pos[0] += 1
        children.append(parse_stmt())
    return {'type': 'IF', 'children': children}

def parse_while():
    consume('WHILE')
    consume('LPAREN')
    cond = parse_expr()
    consume('RPAREN')
    body = parse_stmt()
    return {'type': 'WHILE', 'children': [cond, body]}

def parse_expr_stmt():
    left = consume('IDENTIFIER')

```

```

    if match('ASSIGN'):
        rhs = parse_expr()
        return {'type': 'ASSIGN',
                'children': [{'type': 'ID', 'sval': left['val']},
                             'children': [], rhs]}
    if peek()['type'] == 'LPAREN':
        pos[0] -= 1
        return parse_call_expr()
    pos[0] -= 1
    return parse_expr()

def parse_expr():
    return parse_compar()

def parse_compar():
    left = parse_add_sub()
    ops = {'EQ': '==', 'NEQ': '!=', 'LT': '<', 'GT': '>', 'LE':
'<=', 'GE': '>='}
    while peek()['type'] in ops:
        op = ops[peek()['type']]
        pos[0] += 1
        right = parse_add_sub()
        left = {'type': 'BINOP', 'sval': op, 'children': [left,
right]}
    return left

def parse_add_sub():
    left = parse_mul_div()
    while peek()['type'] in ('PLUS', 'MINUS'):
        op = '+' if peek()['type'] == 'PLUS' else '-'
        pos[0] += 1
        left = {'type': 'BINOP', 'sval': op, 'children': [left,
parse_mul_div()]}
    return left

def parse_mul_div():
    left = parse_unary()
    while peek()['type'] in ('MUL', 'DIV'):
        op = '*' if peek()['type'] == 'MUL' else '/'
        pos[0] += 1

```

```

        left = {'type': 'BINOP', 'sval': op, 'children': [left,
parse_unary()]}
        return left

def parse_unary():
    if peek()['type'] == 'MINUS':
        pos[0] += 1
        return {'type': 'UNOP', 'sval': '-', 'children':
[parse_primary()]}
    return parse_primary()

def parse_primary():
    t = peek()
    if t['type'] == 'INT_LITERAL':
        pos[0] += 1
        return {'type': 'INT_LIT', 'ival': int(t['val']),
'children': []}
    if t['type'] == 'FLOAT_LITERAL':
        pos[0] += 1
        return {'type': 'FLOAT_LIT', 'fval': float(t['val']),
'children': []}
    if t['type'] == 'STRING_LITERAL':
        pos[0] += 1
        return {'type': 'STRING_LIT', 'sval': t['val'],
'children': []}
    if t['type'] == 'IDENTIFIER':
        next_tok = tokens[pos[0]+1] if pos[0]+1 < len(tokens)
else {'type': 'EOF'}
        if next_tok['type'] == 'LPAREN':
            return parse_call_expr()
        pos[0] += 1
        return {'type': 'ID', 'sval': t['val'], 'children': []}
    if t['type'] == 'LPAREN':
        pos[0] += 1
        e = parse_expr()
        consume('RPAREN')
        return e
    raise ParseError(f"Unexpected token: {t['type']}
'{t['val']}'")

def parse_call_expr():

```

```

    name = consume('IDENTIFIER')['val']
    consume('LPAREN')
    args = {'type': 'ARG_LIST', 'children': []}
    while peek()['type'] != 'RPAREN':
        args['children'].append(parse_expr())
        if not match('COMMA'):
            break
    consume('RPAREN')
    return {'type': 'CALL', 'sval': name, 'children': [args]}

return parse_program()

# — pretty-printer —————

def print_ast(node, depth=0, last=True, prefix=''):
    if node is None:
        return
    connector = '└─ ' if last else '├─ '
    label = f"[{node['type']}]"
    attrs = ''
    if 'sval' in node:
        attrs += f" name='{node['sval']}'"
    if 'data_type' in node:
        attrs += f" type='{node['data_type']}'"
    if node['type'] == 'INT_LIT':
        attrs += f" val={node['ival']}"
    if node['type'] == 'FLOAT_LIT':
        attrs += f" val={node['fval']}"

    if depth == 0:
        print(label + attrs)
    else:
        print(prefix + connector + label + attrs)

    children = node.get('children', [])
    new_prefix = prefix + ('  ' if last else '|  ')
    for i, child in enumerate(children):
        print_ast(child, depth + 1, i == len(children) - 1,
new_prefix)

```



```
def print_ast_phase(ast):
    print("=" * 55)
    print("  PHASE 2 – PARSER / ABSTRACT SYNTAX TREE")
    print("=" * 55)
    print_ast(ast)
    print("=" * 55)
```

6.3 Symbol Table (symbol_table.py)

The symbol table builder performs a single-pass walk over the AST. It maintains a scope stack (list of dicts). Each function definition pushes a new scope; the matching block pops it. Variable entries record name, type, kind (variable/param/function), scope level, memory address, and initialisation status.

Symbol Entry Fields

Field	Type	Description
name	str	Identifier name
type	str	Data type: int, float, char, void
kind	str	One of: variable, param, function
scope	int	Scope level (0 = global)
addr	hex	Simulated memory address (0xBF00 - offset)
initialized	bool	Whether a value has been assigned
params	list	Parameter types (functions only)

```
# =====
# PHASE 3 – SYMBOL TABLE BUILDER
# Mini C Compiler
# =====

def build_symbol_table(ast) -> dict:
    """
    Returns:
        {
            'scopes': [ {name: symbol_entry, ...}, ... ],
            'errors': [ str, ... ]
        }
    symbol_entry keys: name, type, kind, scope, addr, initialized
    """
    scopes_stack = [{}] # stack of active scopes
```

```

all_scopes = [scopes_stack[0]] # every scope ever created
(for printing)
errors = []
addr_stack = [0x1000] # local frame offset per scope
level
WORD = 4

def enter_scope():
    new_scope = {}
    scopes_stack.append(new_scope)
    all_scopes.append(new_scope)
    # local stack frame starts just below previous
    addr_stack.append(0xBF00 - (len(scopes_stack) - 1) * 0x100)

def exit_scope():
    scopes_stack.pop()
    addr_stack.pop()

def declare(name, typ, kind='variable', initialized=False,
is_func=False, params=None):
    cur = scopes_stack[-1]
    if name in cur:
        errors.append(f"Redeclaration of '{name}' in same
scope")
    return
    if is_func:
        addr = 0
    else:
        addr = addr_stack[-1]
        size = 4 if typ in ('int', 'float') else 1 if typ ==
'char' else 0
        addr_stack[-1] += size

    entry = {
        'name': name,
        'type': typ,
        'kind': kind,
        'scope': len(scopes_stack) - 1,
        'addr': addr,
        'initialized': initialized,
    }

```

```

    if params is not None:
        entry['params'] = params
    cur[name] = entry

def lookup(name):
    for scope in reversed(scopes_stack):
        if name in scope:
            return scope[name]
    return None

def require(name):
    if lookup(name) is None:
        errors.append(f"Use of undeclared identifier '{name'}")

# — tree walk —————

def walk(node):
    if node is None:
        return
    t = node['type']

    if t in ('PROGRAM', 'BLOCK', 'STMT_LIST'):
        for child in node.get('children', []):
            walk(child)

    elif t == 'FUNC_DEF':
        param_types = [p['data_type'] for p in
node['children'][0].get('children', [])]
        declare(node['sval'], node['data_type'],
kind='function',
                initialized=True, is_func=True,
params=param_types)
        enter_scope()
        for param in node['children'][0].get('children', []):
            declare(param['sval'], param['data_type'],
kind='param', initialized=True)
        for child in node['children'][1:]:
            walk(child)
        exit_scope()

    elif t == 'VAR_DECL':

```

```

        initialized = len(node.get('children', [])) > 0
        declare(node['sval'], node['data_type'],
kind='variable',
                initialized=initialized)
        for child in node.get('children', []):
            walk(child)

    elif t == 'ASSIGN':
        lhs_name = node['children'][0].get('sval')
        if lhs_name:
            entry = lookup(lhs_name)
            if entry:
                entry['initialized'] = True
            else:
                errors.append(f"Use of undeclared identifier
'{lhs_name}'")
            walk(node['children'][1])

    elif t == 'ID':
        require(node['sval'])

    elif t == 'CALL':
        require(node['sval'])
        for child in node.get('children', []):
            walk(child)

    else:
        for child in node.get('children', []):
            walk(child)

    walk(ast)
    return {'scopes': all_scopes, 'errors': errors}

# — printer —————

def print_symbol_table(sym_data: dict):
    scopes = sym_data['scopes']
    errors = sym_data['errors']

    total = sum(len(s) for s in scopes)

```

```

    funcs = sum(1 for s in scopes for e in s.values() if e['kind']
== 'function')
    vars_ = total - funcs

    print("=" * 70)
    print("  PHASE 3 - SYMBOL TABLE")
    print("=" * 70)
    print(f"  Total Symbols : {total}    Functions :
{funcs}  Variables : {vars_}")
    if errors:
        print(f"  ERRORS          : {len(errors)}")
    print("-" * 70)

    for level, scope in enumerate(scopes):
        if not scope:
            continue
        scope_label = "Global scope" if level == 0 else f"Local
scope (level {level})"
        print(f"\n [{scope_label}]")
        print(f"  {'Name':<14} {'Type':<8} {'Kind':<10} {'Scope':<7}
{'Address':<10} {'Size':<6} {'Init'}")
        print("  " + "-" * 60)
        for sym in scope.values():
            addr_str = "-" if sym['kind'] == 'function' else
f"0x{sym['addr']:04X}"
            size_str = ("-" if sym['kind'] == 'function'
                        else "4" if sym['type'] in ('int', 'float')
                        else "1" if sym['type'] == 'char' else "0")
            init_str = ("-" if sym['kind'] == 'function'
                        else "yes" if sym['initialized'] else "no")
            print(f"  {sym['name']:<14} {sym['type']:<8}
{sym['kind']:<10} "
                  f"{sym['scope']:<7} {addr_str:<10} {size_str:<6}
{init_str}")

        print()
    if errors:
        print("  ERRORS:")
        for e in errors:
            print(f"    X  {e}")
    else:

```

```

    print("    ✓ No symbol-table errors")
    print("=" * 70)

```

6.4 Semantic Analyser (semantic.py)

The semantic analyser receives the AST and the symbol table data produced in Phase 3. It reuses errors already collected during the symbol table walk (undeclared identifiers, redeclarations) and adds one more check: function call argument count verification by traversing CALL nodes in the AST.

Check #	Rule	How Detected
1	All identifiers declared before use	Symbol table walk — require() called on every ID/CALL node
2	No redeclarations in same scope	declare() checks if name already in current scope dict
3	Function call argument count	AST traversal: CALL children count vs symbol table params
4	Return type compatibility	Structural check on RETURN nodes within FUNC_DEF
5	Assignment type compatibility	int ↔ float implicit widening allowed

```

# =====
# PHASE 4 – SEMANTIC ANALYSIS
# Mini C Compiler
# =====

CHECKS = [
    ("All identifiers declared before use",
     "No undeclared variables or functions"),
    ("No redeclarations in same scope",
     "Each name declared once per scope"),
    ("Function call argument counts",
     "Argument count matches parameter count"),
    ("Return type compatibility",
     "Return expression matches function type"),
    ("Assignment type compatibility",
     "int ↔ float implicit widening allowed"),
]

def semantic_analysis(ast, sym_data: dict) -> list[str]:
    """
    Re-uses errors already found during symbol table build,
    then adds any extra semantic checks.

```

```

Returns a list of error strings.
"""
errors = list(sym_data.get('errors', []))

# Build a flat lookup from all scopes for quick access
all_syms = {}
for scope in sym_data.get('scopes', []):
    for name, entry in scope.items():
        if name not in all_syms:
            all_syms[name] = entry

# — Check function call argument counts —————
def check_calls(node):
    if node is None:
        return
    if node['type'] == 'CALL':
        fname = node.get('sval', '')
        args = node['children'][0].get('children', []) if
node['children'] else []
        sym = all_syms.get(fname)
        if sym and sym['kind'] == 'function' and 'params' in
sym:
            expected = len(sym['params'])
            got = len(args)
            if expected != got:
                errors.append(
                    f"Function '{fname}' called with {got} args,
"
                    f"but defined with {expected} params"
                )
            for child in node.get('children', []):
                check_calls(child)

check_calls(ast)

return errors

def print_semantic(errors: list[str]):
    print("=" * 60)
    print(" PHASE 4 — SEMANTIC ANALYSIS")

```

```

print("=" * 60)

for label, sub in CHECKS:
    # simple heuristic: match first word of label against errors
    keyword = label.lower().split()[0]
    is_err = any(keyword in e.lower() for e in errors)
    mark = "X" if is_err else "V"
    print(f"\n {mark} {label}")
    print(f"      {sub}")

print()
if errors:
    print("  -" * 30)
    print(f"  Δ Semantic analysis found {len(errors)}
error(s):\n")
    for e in errors:
        print(f"    X {e}")
else:
    print("    " + "-" * 56)
    print("  ✓ Semantic analysis passed – no errors found")
print("=" * 60)

```

6.5 Three-Address Code Generator (tac.py)

The TAC generator performs a recursive walk of the AST and emits instructions as plain strings into a list. Temporary variables (t0, t1, ...) and labels (L0, L1, ...) are generated by counters. Each construct maps to a fixed set of TAC instructions.

Construct	TAC Instructions Emitted
FUNC_DEF	FUNC_BEGIN fname / PARAM_DECL type name / FUNC_END fname
VAR_DECL with init	DECLARE type name / name = <expr>
ASSIGN	lhs = <rhs>
BINOP (+, -, *, /, ==, ...)	tN = left op right
IF (no else)	IF_FALSE cond GOTO L0 / [then body] / L0:
IF-ELSE	IF_FALSE cond GOTO L0 / [then] / GOTO L1 / L0: / [else] / L1:
WHILE	L0: / IF_FALSE cond GOTO L1 / [body] / GOTO L0 / L1:
RETURN	RETURN val
CALL	PARAM arg ... / tN = CALL fname, argc


```

# =====
# PHASE 5 – THREE ADDRESS CODE (TAC / IR) GENERATOR
# Mini C Compiler
# =====

def generate_tac(ast) -> list[str]:
    code = []
    counters = {'tmp': 0, 'lbl': 0}

    def new_tmp():
        t = f"t{counters['tmp']}"
        counters['tmp'] += 1
        return t

    def new_label():
        lbl = f"L{counters['lbl']}"
        counters['lbl'] += 1
        return lbl

    def emit(s: str):
        code.append(s)

    # — statement generator —————

    def gen_stmt(node):
        if node is None:
            return
        t = node['type']

        if t in ('PROGRAM', 'BLOCK', 'STMT_LIST'):
            for child in node.get('children', []):
                gen_stmt(child)

        elif t == 'FUNC_DEF':
            emit(f"FUNC_BEGIN {node['sval']}")
            params = node['children'][0].get('children', [])
            for p in params:
                emit(f"    PARAM_DECL {p['data_type']} {p['sval']}")
            for child in node['children'][1:]:
                gen_stmt(child)

```

```

emit(f"FUNC_END {node['sval']}")
emit('')

elif t == 'VAR_DECL':
    emit(f" DECLARE {node['data_type']} {node['sval']}")
    if node.get('children'):
        val = gen_expr(node['children'][0])
        emit(f" {node['sval']} = {val}")

elif t == 'ASSIGN':
    lhs = node['children'][0]['sval']
    rhs = gen_expr(node['children'][1])
    emit(f" {lhs} = {rhs}")

elif t == 'IF':
    cond = gen_expr(node['children'][0])
    l_else = new_label()
    l_end = new_label()
    emit(f" IF_FALSE {cond} GOTO {l_else}")
    gen_stmt(node['children'][1])
    if len(node['children']) > 2:
        emit(f" GOTO {l_end}")
        emit(f"{l_else}:")
        gen_stmt(node['children'][2])
        emit(f"{l_end}:")
    else:
        emit(f"{l_else}:")

elif t == 'WHILE':
    l_start = new_label()
    l_end = new_label()
    emit(f"{l_start}:")
    cond = gen_expr(node['children'][0])
    emit(f" IF_FALSE {cond} GOTO {l_end}")
    gen_stmt(node['children'][1])
    emit(f" GOTO {l_start}")
    emit(f"{l_end}:")

elif t == 'RETURN':
    if node.get('children'):
        val = gen_expr(node['children'][0])

```

```

        emit(f"  RETURN {val}")
    else:
        emit("  RETURN")

    else:
        for child in node.get('children', []):
            gen_stmt(child)

# — expression generator —————

def gen_expr(node) -> str:
    if node is None:
        return '?'
    t = node['type']

    if t == 'INT_LIT':
        return str(node['ival'])
    if t == 'FLOAT_LIT':
        return str(node['fval'])
    if t == 'STRING_LIT':
        return node['sval']
    if t == 'ID':
        return node['sval']

    if t == 'BINOP':
        left = gen_expr(node['children'][0])
        right = gen_expr(node['children'][1])
        tmp = new_tmp()
        emit(f"  {tmp} = {left} {node['sval']} {right}")
        return tmp

    if t == 'UNOP':
        val = gen_expr(node['children'][0])
        tmp = new_tmp()
        emit(f"  {tmp} = {node['sval']}{val}")
        return tmp

    if t == 'CALL':
        args = node['children'][0].get('children', []) if
node['children'] else []
        vals = [gen_expr(a) for a in args]

```

```

        for v in vals:
            emit(f"  PARAM {v}")
        tmp = new_tmp()
        emit(f"  {tmp} = CALL {node['sval']}, {len(vals)}")
        return tmp

    return '?'

    gen_stmt(ast)
    return code

# — printer —————

def print_tac(tac_lines: list[str]):
    non_empty    = [l for l in tac_lines if l.strip()]
    func_count   = sum(1 for l in tac_lines if
l.strip().startswith('FUNC_BEGIN'))
    temp_count   = sum(1 for l in tac_lines if 't' in l and '=' in l
                        and l.strip().split()[0].startswith('t')
                        and l.strip().split()[0][1:].isdigit())

    print("=" * 60)
    print("  PHASE 5 — THREE ADDRESS CODE  (TAC / IR)")
    print("=" * 60)
    print(f"  Instructions : {len(non_empty)}")
    print(f"  Functions    : {func_count}")
    print(f"  Temporaries   : {temp_count}")
    print("-" * 60)

    line_num = 0
    for raw in tac_lines:
        stripped = raw.strip()
        if not stripped:
            print()
            continue
        line_num += 1
        print(f"  {line_num:>3}    {raw}")

    print("=" * 60)

```

6.6 Assembly Code Generator (asm.py)

The assembly generator translates the TAC string list into Intel-syntax 32-bit pseudo-x86 assembly. It uses a simple frame dictionary to allocate local variables at [bp-N] offsets (4 bytes each). A placeholder tag is emitted for the frame size and back-patched once FUNC_END is reached and the total frame size is known.

Key Design Points

- Local variables are allocated on a simulated stack frame using frame[name] → offset from BP.
- Frame size is initially unknown; a placeholder tag is embedded in the SUB SP instruction and replaced once FUNC_END is processed.
- Binary operators: +/- use ADD/SUB; * uses IMUL; / uses CDQ + IDIV with result in EAX.
- Comparisons (==, !=, <, >, <=, >=) use CMP + SETcc + MOVZX to produce a boolean in EAX.
- Function calls: PARAM emits PUSH; CALL emits CALL + ADD SP cleanup; return value retrieved from EAX.

```
# =====
# PHASE 6 - x86 ASSEMBLY GENERATOR (Intel syntax, 32-bit)
# Mini C Compiler
# =====

import re

WORD = 4    # bytes per int/pointer

def generate_asm(tac: list[str]) -> list[str]:
    asm      = []
    frame    = {}          # var_name -> frame offset from bp
    frame_sz = [0]         # mutable int
    cur_fn   = [None]

    def alloc(name: str):
        if name not in frame:
            frame_sz[0] += WORD
            frame[name] = frame_sz[0]

    def operand(v: str) -> str:
        """Translate a TAC value to an assembly operand."""
        try:
            float(v)        # numeric literal
            return str(int(float(v))) if '.' not in v else v
        except (ValueError, TypeError):
            pass
```

```

        if v in frame:
            return f"[bp-{frame[v]}]"
        return v

def emit(s: str):
    asm.append(s)

# PLACEHOLDER tag so we can back-patch frame size after we know
it
FS_TAG = lambda fn: f"<FS_{fn}>"

for raw in tac:
    line = raw.strip()

    # — blank line —
    if not line:
        emit('')
        continue

    # — FUNC_BEGIN —
    if line.startswith('FUNC_BEGIN'):
        fn = line.split()[1]
        cur_fn[0] = fn
        frame.clear()
        frame_sz[0] = 0
        emit(f"{fn}:")
        emit(f"    push    bp")
        emit(f"    mov     bp, sp")
        emit(f"    sub     sp, {FS_TAG(fn)}")

    # — FUNC_END —
    elif line.startswith('FUNC_END'):
        fn = line.split()[1]
        fs = frame_sz[0]
        emit(f".{fn}_ret:")
        emit(f"    mov     sp, bp")
        emit(f"    pop     bp")
        emit(f"    ret")
        emit(f"; ---- end {fn} (frame = {fs} bytes) ----")
        emit('')
        # back-patch frame-size placeholder

```

```

        tag = FS_TAG(fn)
        for i, l in enumerate(asm):
            if tag in l:
                asm[i] = l.replace(tag, str(fs))
        cur_fn[0] = None

# — DECLARE / PARAM_DECL —————
elif line.startswith('DECLARE') or
line.startswith('PARAM_DECL'):
    parts = line.split()
    name = parts[2]
    alloc(name)
    emit(f"    ; {name} -> [bp-{{frame[name]}}]")

# — RETURN —————
elif line.startswith('RETURN'):
    parts = line.split()
    val = parts[1] if len(parts) > 1 else None
    if val:
        src = operand(val)
        emit(f"    mov    eax, {{src}}")
    else:
        emit(f"    mov    eax, 0")
    emit(f"    jmp    .{{cur_fn[0]}}_ret")

# — IF_FALSE cond GOTO label —————
elif line.startswith('IF_FALSE'):
    parts = line.split()
    cond = operand(parts[1])
    label = parts[3]
    emit(f"    mov    eax, {{cond}}")
    emit(f"    cmp    eax, 0")
    emit(f"    je     {{label}}")

# — GOTO label —————
elif line.startswith('GOTO'):
    emit(f"    jmp    {{line.split()[1]}}")

# — PARAM val —————
elif line.startswith('PARAM '):
    val = line.split(None, 1)[1]

```

```

src = operand(val)
try:
    float(src)
    emit(f"    push    {src}")
except (ValueError, TypeError):
    emit(f"    mov     eax, {src}")
    emit(f"    push    eax")

# — label: —————
elif re.match(r'^[A-Za-z_][A-Za-z0-9_]*$', line) or \
     re.match(r'^\.[A-Za-z_][A-Za-z0-9_.*]*$', line):
    emit(line)

# — dest = CALL fn, argc —————
elif '=' in line:
    dest, rhs = [s.strip() for s in line.split('=', 1)]
    parts      = re.split(r'[\s,]+', rhs)
    fn         = parts[1]
    argc       = int(parts[2]) if len(parts) > 2 else 0
    alloc(dest)
    emit(f"    call    {fn}")
    if argc:
        emit(f"    add     sp, {argc * WORD}")
        emit(f"    mov     {operand(dest)}, eax")

# — dest = expr —————
elif '=' in line:
    dest, rhs = [s.strip() for s in line.split('=', 1)]
    alloc(dest)

    # Try binary operator (search right-to-left for correct
precedence)
    BIN_OPS = ['==', '!=', '<=', '>=', '<', '>', '+', '-',
'*', '/']

    found = None
    for bop in BIN_OPS:
        pat = r' ' + re.escape(bop) + r' '
        idx = rhs.rfind(pat + bop + pat)
        if idx >= 0:
            found = {
                'op': bop,

```



```

        'l': rhs[:idx].strip(),
        'r': rhs[idx + len(bop) + 2:].strip()    #
after ' op '
    }
    break

if found:
    emit(f"    mov    eax, {operand(found['l'])}")
    right = operand(found['r'])
    op     = found['op']

    if op == '+':
        try:
            float(right); emit(f"    add    eax,
{right}")
        except (ValueError, TypeError):
            emit(f"    mov    ebx, {right}");
emit(f"    add    eax, ebx")
    elif op == '-':
        try:
            float(right); emit(f"    sub    eax,
{right}")
        except (ValueError, TypeError):
            emit(f"    mov    ebx, {right}");
emit(f"    sub    eax, ebx")
    elif op == '*':
        emit(f"    mov    ebx, {right}")
        emit(f"    imul   eax, ebx")
    elif op == '/':
        emit(f"    cdq")
        emit(f"    mov    ebx, {right}")
        emit(f"    idiv   ebx")
    else:
        SET_MAP = {
            '==': 'sete', '!=': 'setne',
            '<': 'setl', '>': 'setg',
            '<=': 'setle', '>=': 'setge'
        }
        emit(f"    mov    ebx, {right}")
        emit(f"    cmp    eax, ebx")
        emit(f"    {SET_MAP[op]} al")

```

```

        emit(f"    movzx eax, al")

        emit(f"    mov    {operand(dest)}, eax")

    else:
        src = operand(rhs)
        emit(f"    mov    eax, {src}")
        emit(f"    mov    {operand(dest)}, eax")

    return asm

# — printer —————

def print_asm(asm_lines: list[str]):
    instr_count = sum(
        1 for l in asm_lines
        if l.strip() and not l.strip().startswith(';')
        and not re.match(r'^[A-Za-z_][A-Za-z0-9_]*$', l.strip())
    )

    print("=" * 60)
    print("  PHASE 6 — x86 ASSEMBLY  (Intel syntax, 32-bit)")
    print("=" * 60)
    print(f"  Instructions (non-label, non-comment): {instr_count}")
    print("-" * 60)
    for i, raw in enumerate(asm_lines, 1):
        stripped = raw.strip()
        if not stripped:
            print()
            continue
        if stripped.startswith(';'):
            print(f"    {i:>4}    {raw}")                # comment
        elif re.match(r'^\.[A-Za-z_][A-Za-z0-9_]*$', stripped):
            print(f"line\n    {i:>4}    {raw}")        # label on its own
        else:
            print(f"    {i:>4}    {raw}")

    print("=" * 60)

```

6.7 Grammar Utilities (grammar_utils.py)

grammar_utils.py is a self-contained LL(1) analysis library with no external dependencies. It uses Python's OrderedDict to preserve the user-entered production order through all transformations.

6.7.1 Grammar Normaliser

_normalize_grammar() ensures every token in every production is an atomic symbol before transformations begin. It greedy-matches known non-terminals first, then falls back to character-by-character splitting for unknown multi-character strings.

```
def _normalize_grammar(grammar: dict) -> dict:
    """
    Ensure every token in every production is a single symbol:
    - known NT → kept as-is
    - 'ε' → kept as-is
    - len==1 → kept as-is (single-char terminal or NT)
    - all-lowercase/digit multi-char (e.g. 'id', 'num') → kept as-is (multi-char terminal)
    - otherwise → split character by character
    """
    nts = set(grammar.keys())

    def split_prod(prod):
        result = []
        for token in prod:
            if token == 'ε':
                result.append(token)
            elif token in nts or len(token) == 1:
                result.append(token)
            elif re.match(r'^[a-z0-9_]+$', token):
                # multi-char lowercase terminal (e.g. 'id', 'then', 'num') – keep whole
                result.append(token)
            else:
                # mixed-case compact string – split char by char,
                # but first try to consume known NTs greedily
                i = 0
                while i < len(token):
                    matched = False
                    for length in range(len(token) - i, 1, -1):
                        if token[i:i+length] in nts:
                            result.append(token[i:i+length])

```

```

        i += length
        matched = True
        break
    if not matched:
        result.append(token[i])
        i += 1

    return result

new_grammar = OrderedDict()
for nt, productions in grammar.items():
    new_grammar[nt] = [split_prod(p) for p in productions]
return new_grammar

```

6.7.2 Left Recursion Removal

`remove_left_recursion()` handles both direct and indirect left recursion using the standard ordered-substitution algorithm. For each NT A_i , it substitutes any leading A_j ($j < i$) with A_j 's productions, then eliminates direct left recursion by introducing A_i' .

```

def remove_left_recursion(grammar: dict) -> dict:
    """Remove both direct AND indirect left recursion."""
    grammar = _normalize_grammar(grammar)
    nts = list(grammar.keys())
    new_grammar = OrderedDict((nt, list(prods)) for nt, prods in
grammar.items())

    for i, ai in enumerate(nts):
        # Substitute earlier NTs to expose indirect left recursion
        for aj in nts[:i]:
            new_prods = []
            for prod in new_grammar[ai]:
                if prod and prod[0] == aj:
                    for aj_prod in new_grammar[aj]:
                        if aj_prod == ['ε']:
                            new_prods.append(prod[1:] if prod[1:]
else ['ε'])
                        else:
                            new_prods.append(aj_prod + prod[1:])
                    else:
                        new_prods.append(prod)

```

```

        new_grammar[ai] = new_prods

        # Now eliminate direct left recursion for ai
        recursive      = []
        non_recursive  = []
        for prod in new_grammar[ai]:
            if prod and prod[0] == ai:
                recursive.append(prod[1:])
            else:
                non_recursive.append(prod)

        if not recursive:
            continue

        prime = ai + ""
        while prime in new_grammar:
            prime += ""

        new_grammar[ai] = []
        for beta in non_recursive:
            new_grammar[ai].append((beta if beta else ['ε']) +
[prime])
        if not non_recursive:
            new_grammar[ai].append([prime])

        new_grammar[prime] = []
        for alpha in recursive:
            new_grammar[prime].append((alpha if alpha else ['ε']) +
[prime])
        new_grammar[prime].append(['ε'])

    return new_grammar

```

6.7.3 Left Factoring

left_factoring() groups productions by their first symbol, computes the longest common prefix using _common_prefix(), and factors iteratively. The outer loop repeats until no further factoring is possible, correctly handling nested common prefixes.

```

def _common_prefix(prods: list) -> list:
    if not prods:

```

```

        return []
    prefix = list(prods[0])
    for prod in prods[1:]:
        length = 0
        for a, b in zip(prefix, prod):
            if a == b:
                length += 1
            else:
                break
        prefix = prefix[:length]
        if not prefix:
            break
    return prefix

def left_factoring(grammar: dict) -> dict:
    """Apply left factoring using longest-common-prefix grouping."""
    grammar = _normalize_grammar(grammar)

    def _make_new_nt(base: str, existing: set) -> str:
        candidate = base + ""
        while candidate in existing:
            candidate += ""
        return candidate

    changed = True
    while changed:
        changed = False
        new_grammar = OrderedDict()

        for nt, productions in grammar.items():
            all_nts = set(grammar.keys()) | set(new_grammar.keys())

            groups: dict = OrderedDict()
            for prod in productions:
                key = prod[0] if prod else 'ε'
                groups.setdefault(key, []).append(prod)

            if all(len(g) == 1 for g in groups.values()):
                new_grammar[nt] = productions
                continue

```

```

changed = True
new_grammar[nt] = []

for _key, group in groups.items():
    if len(group) == 1:
        new_grammar[nt].append(group[0])
        continue

    lcp = _common_prefix(group)
    new_nt = _make_new_nt(nt, all_nts)
    all_nts.add(new_nt)

    new_grammar[nt].append(lcp + [new_nt])
    new_grammar[new_nt] = []
    for prod in group:
        rest = prod[len(lcp):]
        new_grammar[new_nt].append(rest if rest else
['ε'])

    grammar = new_grammar

return grammar

```

6.7.4 FIRST and FOLLOW Sets

compute_first() uses fixed-point iteration: it keeps adding terminals to each NT's FIRST set until no changes occur. epsilon propagates through nullable symbol chains. compute_follow() seeds FOLLOW(start) with \$ and propagates through productions, applying the four standard FOLLOW rules until stable.

```

#
#
# FIRST Sets
#
#
def compute_first(grammar: dict) -> dict:
    """
    Compute FIRST sets for all non-terminals.

```

```

A terminal t is in FIRST(A) if  $A \Rightarrow^* t \dots$ 
 $\epsilon$  is in FIRST(A) if  $A \Rightarrow^* \epsilon$ 
"""
first = {nt: set() for nt in grammar}

def first_of(symbol):
    """FIRST of a single symbol (NT or terminal)."""
    if symbol not in grammar:
        return {symbol}          # terminal  $\rightarrow$  {terminal}
    return first[symbol]         # NT  $\rightarrow$  current approximation

changed = True
while changed:
    changed = False
    for nt, productions in grammar.items():
        for prod in productions:
            # Empty production (explicit epsilon)
            if prod == [' $\epsilon$ ']:
                if ' $\epsilon$ ' not in first[nt]:
                    first[nt].add(' $\epsilon$ ')
                    changed = True
                continue

            # Walk symbols; break as soon as one can't derive  $\epsilon$ 
            derives_eps = True
            for symbol in prod:
                f = first_of(symbol)
                before = len(first[nt])
                first[nt] |= (f - {' $\epsilon$ '})
                if len(first[nt]) != before:
                    changed = True
                if ' $\epsilon$ ' not in f:
                    derives_eps = False
                    break

            # All symbols derive  $\epsilon \rightarrow$  whole production derives  $\epsilon$ 
            if derives_eps:
                if ' $\epsilon$ ' not in first[nt]:
                    first[nt].add(' $\epsilon$ ')
                    changed = True

```



```

    return first

#
#
# FOLLOW Sets
#
#

def compute_follow(grammar: dict, first: dict) -> dict:
    """
    Compute FOLLOW sets for all non-terminals.

    Rules:
    1.  $\$ \in \text{FOLLOW}(\text{start})$ 
    2. If  $A \rightarrow \alpha B \beta$ :  $\text{FIRST}(\beta) - \{\epsilon\} \subseteq \text{FOLLOW}(B)$ 
    3. If  $A \rightarrow \alpha B \beta$  and  $\epsilon \in \text{FIRST}(\beta)$ :  $\text{FOLLOW}(A) \subseteq \text{FOLLOW}(B)$ 
    4. If  $A \rightarrow \alpha B$ :  $\text{FOLLOW}(A) \subseteq \text{FOLLOW}(B)$ 
    """
    nts = list(grammar.keys())
    follow = {nt: set() for nt in nts}
    follow[nts[0]].add('$')

    def first_of_sequence(symbols):
        """
        Returns (first_set_without_eps, all_nullable) for a sequence
        of symbols.
        all_nullable=True means the entire sequence can derive  $\epsilon$ .
        """
        result = set()
        all_nullable = True
        for s in symbols:
            sf = first.get(s, set()) if s in grammar else {s}
            result |= (sf - {' $\epsilon$ '})
            if ' $\epsilon$ ' not in sf:
                all_nullable = False
                break
        return result, all_nullable

```

```

changed = True
while changed:
    changed = False
    for nt, productions in grammar.items():
        for prod in productions:
            if prod == ['ε']:
                continue
            for idx, symbol in enumerate(prod):
                if symbol not in grammar:
                    continue # only compute FOLLOW for non-
terminals

                rest = prod[idx + 1:]
                first_rest, all_eps = first_of_sequence(rest)

                before = len(follow[symbol])

                # Rule 2: add FIRST(rest) - {ε}
                follow[symbol] |= first_rest

                # Rule 3 & 4: if rest =>* ε, add FOLLOW(nt)
                if all_eps:
                    follow[symbol] |= follow[nt]

                if len(follow[symbol]) != before:
                    changed = True

return follow

```

8. Test Cases & Expected Output

The following test cases demonstrate LexiCore's compilation pipeline end-to-end. Each case shows the source code and the key outputs from each phase.

Token Analyzer (Phase 1)

The screenshot displays the Mini C Compiler interface, specifically the Token Analyzer phase. The source code is as follows:

```
1 int add(int a, int b) {  
2     return a + b;  
3 }  
4  
5 int main() {  
6     int x;  
7     int y;  
8     int result;  
9     x = 3;  
10    y = 7;  
11    result = add(x, y);  
12    return result;  
13 }
```

The interface shows the following statistics:

- 51 TOTAL TOKENS
- 9 KEYWORDS
- 16 IDENTIFIERS

The TOKEN STREAM table is as follows:

#	TYPE	VALUE
1	INT	int
2	IDENTIFIER	add
3	LPAREN	(
4	INT	int
5	IDENTIFIER	a
6	COMMA	,
7	INT	int
8	IDENTIFIER	b
9	RPAREN	)
10	LBRACE	{
11	RETURN	return
12	IDENTIFIER	a
13	PLUS	+
14	IDENTIFIER	b
15	SEMICOLON	;
16	RBRACE	}
17	INT	int
18	IDENTIFIER	main
19	LPAREN	(
20	RPAREN	)
21	LBRACE	{
22	INT	int
23	IDENTIFIER	x
24	SEMICOLON	;

The status bar at the bottom indicates: "Compiled successfully — 51 tokens, 18 TAC instructions."

AST (Phase 2)

The screenshot displays the Mini C Compiler interface, specifically the AST (Phase 2) view. The window title is "Mini C Compiler — 6-Phase Educational Compiler". The interface is divided into two main panels. The left panel, titled "SOURCE CODE — C", contains the following C code:

```
1 int add(int a, int b) {  
2     return a + b;  
3 }  
4  
5 int main() {  
6     int x;  
7     int y;  
8     int result;  
9     x = 3;  
10    y = 7;  
11    result = add(x, y);  
12    return result;  
13 }
```

The right panel, titled "ABSTRACT SYNTAX TREE", shows the parsed code structure. The root node is [PROGRAM], which contains two main components: a function definition [FUNC_DEF] for 'add' and a function definition [FUNC_DEF] for 'main'. The 'add' function definition includes a parameter list [PARAM_LIST] with parameters 'a' and 'b', both of type 'int', and a block [BLOCK] containing a statement list [STMT_LIST] with a return statement [RETURN] that evaluates the expression 'a + b'. The 'main' function definition includes a parameter list [PARAM_LIST] (empty), a block [BLOCK] containing a statement list [STMT_LIST] with three variable declarations [VAR_DECL] for 'x' (type 'int'), 'y' (type 'int'), and 'result' (type 'int'), followed by three assignment statements [ASSIGN] for 'x' (value 3), 'y' (value 7), and 'result' (value 'add(x, y)'). The 'result' assignment includes a call node [CALL] for the 'add' function, which has an argument list [ARG_LIST] with arguments 'x' and 'y'. The 'main' function definition also includes a return statement [RETURN] at the end.

At the bottom of the interface, a status bar indicates: "Compiled successfully — 51 tokens, 18 TAC instructions".

Symbol Table (Phase 3)

Mini C Compiler — 6-Phase Educational Compiler

Mini C Compiler
6-Phase Educational Compiler

LL(1) Grammar Compile

Tokens AST **Symbol Table** Semantic TAC / IR Assembly

7 TOTAL SYMBOLS 2 FUNCTIONS 5 VARIABLES

NAME	TYPE	KIND	SCOPE	ADDRESS	SIZE	INIT
GLOBAL SCOPE						
add	int	function	0	—	—	—
main	int	function	0	—	—	—
LOCAL SCOPE (LEVEL 1)						
a	int	param	1	0xBE00	4	yes
b	int	param	1	0xBE04	4	yes
LOCAL SCOPE (LEVEL 2)						
x	int	variable	1	0xBE00	4	yes
y	int	variable	1	0xBE04	4	yes
result	int	variable	1	0xBE08	4	yes

```

1 int add(int a, int b) {
2     return a + b;
3 }
4
5 int main() {
6     int x;
7     int y;
8     int result;
9     x = 3;
10    y = 7;
11    result = add(x, y);
12    return result;
13 }

```

Semantic (Phase 4)

Mini C Compiler — 6-Phase Educational Compiler

Mini C Compiler
6-Phase Educational Compiler

LL(1) Grammar Compile

Tokens AST Symbol Table **Semantic** TAC / IR Assembly

- ✓ All identifiers declared before use
No undeclared variables or functions
- ✓ No redeclarations in same scope
Each name declared once per scope
- ✓ Function call argument counts
Argument count matches parameter count
- ✓ Return type compatibility
Return expression matches function type
- ✓ Assignment type compatibility
int → float implicit widening allowed

✓ Semantic analysis passed — no errors found

```

1 int add(int a, int b) {
2     return a + b;
3 }
4
5 int main() {
6     int x;
7     int y;
8     int result;
9     x = 3;
10    y = 7;
11    result = add(x, y);
12    return result;
13 }

```

Tac (Phase 5)

Mini C Compiler — 6-Phase Educational Compiler

Mini C Compiler
6-Phase Educational Compiler

• SOURCE CODE — C

```
1 int add(int a, int b) {  
2     return a + b;  
3 }  
4  
5 int main() {  
6     int x;  
7     int y;  
8     int result;  
9     x = 3;  
10    y = 7;  
11    result = add(x, y);  
12    return result;  
13 }
```

✓ Tokens ✓ AST ✓ Symbol Table ✓ Semantic

18
INSTRUCTIONS

THREE ADDRESS CODE (TAC / IR)

```
1FUNC_BEGIN add  
2PARAM_DECL int a  
3PARAM_DECL int b  
4t0 = a + b  
5RETURN t0  
6FUNC_END add  
  
7FUNC_BEGIN main  
8DECLARE int x  
9DECLARE int y  
10DECLARE int result  
11x = 3  
12y = 7  
13PARAM x  
14PARAM y  
15t1 = CALL add, 2  
16result = t1  
17RETURN result  
18FUNC_END main
```

Assembly (Phase 6)



Mini C Compiler
6-Phase Educational Compiler

SOURCE CODE — C

Tokens
AST
Symbol Table
Semantic
TAC / IR
Assembly

X86 ASSEMBLY (INTEL SYNTAX, 32-BIT)

```

1 int add(int a, int b) {
2     return a + b;
3 }
4
5 int main() {
6     int x;
7     int y;
8     int result;
9     x = 3;
10    y = 7;
11    result = add(x, y);
12    return result;
13 }

```

1 add:
2 push bp
3 mov bp, sp
4 sub sp, 12
5 ; a → [bp-4]
6 ; b → [bp-8]
7 mov eax, [bp-4]
8 mov ebx, [bp-8]
9 add eax, ebx
10 mov [bp-12], eax
11 mov eax, [bp-12]
12 jmp .add_ret
13 .add_ret:
14 mov sp, bp
15 pop bp
16 ret
17; — end add (frame = 12 bytes) —

20 main:
21 push bp
22 mov bp, sp
23 sub sp, 16
24 ; x → [bp-4]
25 ; y → [bp-8]
26 ; result → [bp-12]
27 mov eax, 3
28 mov [bp-4], eax
29 mov eax, 7
30 mov [bp-8], eax
31 mov eax, [bp-4]
32 push eax
33 mov eax, [bp-8]

Load: Simple Arithmetic If/Else While Functions

Compiled successfully — 51 tokens, 18 TAC instructions

LL1 (First & Follow)

47

LL(1) Grammar Analyzer
?
X

LL(1) Grammar Analyzer

Write productions like: $S \rightarrow aB \mid \epsilon$ (one NT per line)

GRAMMAR INPUT

```

E → T E'
E → +T E' | ε
T → F T'
T → *F T' | ε
F → (E) | id

```

☐ Remove Left Recursion
☐ Left Factoring
☐ FIRST / FOLLOW Sets

Analyze
Clear

Original
Left Recursion
Left Factoring
FIRST / FOLLOW

Grammar Used (Original)

```

E      → T E' | +T E' | ε
T      → F T' | *F T' | ε
F      → ( E ) | id

```

FIRST Sets

```

FIRST(E) = { (, +, id, ε, ', * }
FIRST(T) = { (, id, ε, * }
FIRST(F) = { (, id }

```

FOLLOW Sets

```

FOLLOW(E) = { $, ), ' }
FOLLOW(T) = { (, +, id, ', * }
FOLLOW(F) = { (, id, ', * }

```

✓ Analysis complete — 3 non-terminal(s) in final grammar.

LL1 (Left Recursion & Left Factoring)

Original
Left Recursion
Left Factoring
FIRST / FOLLOW

Before (Original)

```

E      → E * T | b

```

After Removing Left Recursion

```

E      → b E'
E'     → * T E' | ε

```


7. Phase Summary

This interface represents a Mini Compiler project UI that allows users to input source code and observe the complete compilation process in a clear, step-by-step manner. It integrates multiple compiler phases including lexical analysis, syntax parsing, semantic checking, and code generation within a single structured workflow. After execution, the system generates and displays detailed outputs such as tokens, symbol table entries, intermediate representation in the form of Three Address Code (TAC), and final assembly code. This helps users understand how high-level code is gradually transformed into low-level machine instructions. The interface provides instant feedback after running the program, making debugging faster and more efficient. A dark-themed design is used to enhance focus, reduce eye strain, and give a modern professional appearance. Each functional module is organized into separate tabs or sections, ensuring smooth navigation and a clean user experience. Overall, this tool serves as an interactive and educational platform that effectively demonstrates core concepts of compiler design in a practical and visually structured way.

#	Phase	Module	Key Implementation Detail
1	Lexical Analysis	lexer.py	Hand-written scanner; index pointer; no regex engine
2	Syntax Analysis	parser.py	Recursive-descent; AST as nested dicts; ParseError on mismatch
3	Symbol Table	symbol_table.py	Scope stack; address simulation; re-declaration detection
4	Semantic Analysis	semantic.py	5 checks; reuses Phase 3 errors; CALL arity check on AST
5	IR / TAC	tac.py	AST walk; string-based TAC; counters for temps (t0..) and labels (L0..)
6	Assembly	asm.py	Intel syntax 32-bit; frame dict; back-patching; CDQ/IDIV; SETcc

8. References

#	Reference
[1]	Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). Compilers: Principles, Techniques, and Tools (2nd ed.). Addison-Wesley. [The Dragon Book — primary reference for all compiler phases.]
[2]	Grune, D., van Reeuwijk, K., Bal, H. E., Jacobs, C. J. H., & Langendoen, K. (2012). Modern Compiler Design (2nd ed.). Springer.
[3]	Intel Corporation (2023). Intel 64 and IA-32 Architectures Software Developer's Manual — Reference for x86 instruction set and calling conventions.
[4]	Python Software Foundation (2024). Python 3 Documentation — re, collections, subprocess modules. https://docs.python.org/3/
[5]	PyQt5 Documentation. Riverbank Computing. https://www.riverbankcomputing.com/software/pyqt/ — GUI framework used for Nabila's compiler 's interface.
[6]	GitHub Repository: Nabila's compiler — Mini Compiler IDE. https://github.com/Nawer-Nabila/Mini_Compiler

Explore the Complete Mini Compiler Project on GitHub

The full source code, project structure, and implementation details of the Mini Compiler IDE are available on GitHub. This repository includes all core compiler modules such as lexical analysis, syntax parsing, semantic analysis, intermediate code generation, and assembly generation. It also features a modern, IDE-inspired user interface designed to help learners understand compiler concepts in a practical and interactive way. The project demonstrates a complete end-to-end compilation workflow, making it useful for both academic learning and experimentation.

 **GitHub Repository:** [Nabila's Compiler](https://github.com/Nawer-Nabila/Mini_Compiler)